

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

Desarrollo Orientado por Objetos (DOPO - POOB) | Periodo 2026-2

INFORME CONSOLIDADO DE RETROSPECTIVAS (CICLOS 1, 2 Y 3)

Proyecto: Simulador Slot Machine y Solución Problem I (49th ICPC World Championship)

Equipo de Desarrollo:	Juan David Espitia y Cristian Salamanca
Proyecto y Contexto:	Simulador Slot Machine & Solución Maratón Mundial ICPC (Problem I)
Dedicación Reportada:	8 Horas/Hombre por integrante en cada ciclo (16 Horas/Hombre en total por ciclo)

El presente documento consolida la retrospectiva metodológica y técnica correspondiente a los Ciclos 1, 2 y 3 del proyecto Slot Machine, desarrollado en el marco de la asignatura Desarrollo Orientado por Objetos (DOPO - POOB) de la Escuela Colombiana de Ingeniería Julio Garavito. El proyecto toma como inspiración el Problem I de la 49ª Maratón Mundial de Programación ICPC 2025 y lo transforma en un ejercicio evolutivo de ingeniería de software orientado por objetos. Como equipo de trabajo, integrado por Juan David Espitia y Cristian Salamanca, estructuramos el desarrollo mediante mini-ciclos iterativos e incrementales. A lo largo de este informe exponemos de manera analítica y reflexiva las decisiones de diseño adoptadas, la evolución de nuestra arquitectura en Astah, los desafíos técnicos afrontados —con especial énfasis en el modelado de diagramas de secuencia y la reutilización de código preexistente—, así como los aprendizajes y compromisos asumidos para las fases de consolidación final.

RETROSPECTIVA CICLO 1: CONSTRUCCIÓN INICIAL DEL SIMULADOR Y BASE EN SHAPES

Alcance: Modelado del dominio en BlueJ, gestión de ruedas y símbolos, rotación, detección de jackpot, alternancia visible/invisible y usabilidad con shapes.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Durante la etapa inicial del proyecto, reconocimos que abordar simultáneamente la concepción de clases, la dinámica de rotación y el renderizado gráfico en BlueJ representaba un riesgo elevado de acoplamiento prematuro. Por consiguiente, como equipo de desarrollo estructuramos el avance del Ciclo 1 en cuatro mini-ciclos secuenciales con objetivos y entregables bien delimitados:

Mini-ciclo	Nombre	Alcance / Requisitos	Justificación Técnica y de Diseño
MC 1.1	Modelado del Dominio y Diseño en Astah	Diagrama de clases inicial definiendo SlotMachine, Wheel y Symbol. Análisis de componentes a reutilizar del paquete shapes.	Fue prioritario establecer la arquitectura de objetos y los contratos de métodos antes de programar, asegurando una adecuada

			distribución de responsabilidades y evitando que SlotMachine absorbiera funciones que pertenecían a las ruedas.
MC 1.2	Lógica del Requerimiento: Ruedas y Símbolos (RF1 - RF3)	Creación de la máquina tragamonedas, adición/eliminación de ruedas (addWheel, delWheel) y gestión de símbolos diferenciados por color (addSymbol, delSymbol).	Garantizar que las estructuras de datos (colecciones y arreglos) preservaran la coherencia del estado interno en memoria (validación de límites e invariantes) con independencia absoluta de la interfaz gráfica.
MC 1.3	Dinámica de Juego y Detección de Jackpot (RF4 - RF6)	Mecanismos de giro (spin), consulta de configuraciones actuales (symbols, configuration) y algoritmo de comprobación del estado ganador (isJackpot).	El giro circular de símbolos y la verificación de victoria constituyen el núcleo de la máquina. Implementar esta lógica de forma aislada permitió verificar matemáticamente el desplazamiento modular de índices y el buen funcionamiento del Jackpot.
MC 1.4	Integración Gráfica y Usabilidad con Shapes (RF7 - RF8)	Control de visibilidad (makeVisible, makeInvisible), terminación (exit), cambio visual ante Jackpot y despliegue de JOptionPane condicionado a la visibilidad activa.	Aislar la interacción con Canvas de shapes al final del ciclo protegió la lógica del dominio de inconsistencias de dibujo y permitió satisfacer la directriz de diseño: el simulador debe ser plenamente operativo en modo invisible.

En síntesis: Esta descomposición en capas lógicas nos permitió progresar de forma metodológica desde la abstracción formal en Astah hacia la estabilidad del modelo en memoria, culminando con la visualización como un observador conmutable. Esto garantizó el principio de extensibilidad exigido por la asignatura desde la primera entrega.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Al término del Ciclo 1, el estado del proyecto alcanzó un buen cumplimiento respecto a los mini-ciclos planificados. Todas las funcionalidades fueron verificadas en BlueJ y los artefactos correspondientes quedaron integrados en el repositorio Git:

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 1.1	Modelado Astah	85% Completado	Diagrama de clases completo con atributos/métodos públicos y privados; diagramas de secuencia de los métodos construidos de SlotMachine deteniendo la traza formalmente en shapes, sin embargo, aún hay detalles puntuales que se necesita pulir
MC 1.2	Lógica Ruedas/Símbolos	100% Completado	Métodos addWheel, delWheel, addSymbol y delSymbol plenamente operativos, gestionando adecuadamente los índices y evitando excepciones de puntero nulo.
MC 1.3	Dinámica y Jackpot	100% Completado	Rotación circular spin() validada; métodos symbols(), configuration() e isJackpot() probados con diversas configuraciones de prueba.
MC 1.4	Integración Shapes & GUI	100% Completado	Alternancia makeVisible() y makeInvisible() funcional; retroalimentación modal con JOptionPane desplegada exclusivamente cuando la máquina está visible.

¿Por qué se encuentra en este estado? Porque se validaron todos los requerimientos funcionales (RF1 a RF8), las restricciones de diseño y los criterios de usabilidad mediante pruebas manuales exhaustivas en el banco de objetos de BlueJ, manteniendo una estricta documentación JavaDoc y respetando la directriz de que ningún método excediera una pantalla de extensión.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

Durante este ciclo registramos una dedicación de 8 Horas/Hombre por integrante, para un total de 16 Horas/Hombre invertidas en equipo. El trabajo se distribuyó equitativamente mediante sesiones conjuntas de análisis y asignación de componentes:

Desarrollador	Horas	Responsabilidades y Foco de Trabajo	Rol en el Ciclo
Juan David Espitia	8 Horas	Modelado de clases y secuencias en Astah, implementación de la lógica algorítmica de rotación circular en Wheel y SlotMachine (MC 1.1 y MC 1.3), y documentación técnica JavaDoc.	Arquitectura y Dominio
Cristian Salamanca	8 Horas	Adaptación de componentes del paquete shapes, control de visibilidad (setVisible/invisible), manejo de excepciones visuales con JOptionPane (MC 1.2 y MC 1.4) y versionamiento en Git.	GUI y Usabilidad

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Como equipo consideramos que nuestro mayor logro en el Ciclo 1 fue haber consolidado una arquitectura orientada a objetos rigurosamente desacoplada, donde la clase principal SlotMachine opera como un orquestador de alto nivel que delega responsabilidades a Wheel y Symbol, permitiendo que el simulador funcione con total autonomía tanto en memoria (modo invisible) como en pantalla a través del paquete shapes.

¿Por qué lo consideramos el mayor logro? Porque en proyectos iniciales de desarrollo de software es muy común cometer el error de mezclar instrucciones de interfaz gráfica (como coordenadas cartesianas y repintado de Canvas) dentro de la lógica del negocio. Haber evitado esa contaminación desde el inicio no solo permitió cumplir con naturalidad los requisitos de usabilidad (como la variación estética en Jackpot y los avisos con JOptionPane), sino que sentó la base técnica que hizo posible incorporar pruebas unitarias automatizadas en el Ciclo 2 y la testing tool en el Ciclo 3 sin necesidad de reescribir el simulador.

■ SÍNTESIS DEL MAYOR LOGRO - CICLO 1

Arquitectura desacoplada: El núcleo del simulador SlotMachine opera con total independencia de la interfaz gráfica.

Reutilización limpia de shapes sin acoplar coordenadas de dibujo dentro de la lógica del juego.

Cumplimiento total de RF1 a RF8, alternancia de visibilidad y retroalimentación visual de estado ganador.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico que afrontamos en el Ciclo 1 involucró dos aspectos interdependientes: la reutilización de código del paquete pedagógico shapes y la elaboración rigurosa de los Diagramas de Secuencia en Astah.

En cuanto a la reutilización de shapes, las clases suministradas (Canvas, Circle, Rectangle) fueron diseñadas para figuras geométricas aisladas. Al intentar modelar una máquina tragamonedas con múltiples ruedas y símbolos polícromos alineados horizontalmente, experimentamos dificultades para calcular las coordenadas relativas y evitar parpadeos visuales al agregar, eliminar o rotar componentes. Además, el requisito de operar en modo invisible exigía garantizar que no se ejecutara ninguna llamada a Canvas cuando la máquina no estuviera visible. Para resolverlo, encapsulamos la responsabilidad visual dentro de cada entidad: Wheel gestiona las coordenadas relativas de sus símbolos y condiciona cualquier redibujado a la bandera isVisible, logrando un rendimiento óptimo y sin parpadeos.

En cuanto a los Diagramas de Secuencia en Astah, el desafío radicó en representar la orquestación de los métodos de SlotMachine respetando la directriz explícita de la guía: 'parar en los componentes de shapes'. Inicialmente tendíamos a incluir en el diagrama toda la fontanería interna de dibujo de Canvas, generando trazas excesivamente densas e ilegibles. Para resolverlo, establecimos a los componentes de shapes como objetos de frontera (boundary objects), modelando únicamente los mensajes dirigidos a sus métodos públicos (makeVisible, draw, erase) y deteniendo la traza allí, lo cual produjo diagramas nítidos, formales y centrados en la lógica del dominio.

Dimensión Técnica	Desafío y Dificultad Afrontada	Estrategia de Solución Aplicada
Reutilización de Código (Paquete shapes)	Desalineación de coordenadas relativas al manipular ruedas dinámicamente y riesgo de invocar Canvas en modo invisible.	Encapsulamiento del renderizado en las entidades del dominio, cálculo de posiciones relativas y control estricto de invocación gráfica mediante la bandera isVisible.
Diagramas de Secuencia (Herramienta Astah)	Sobrecarga de detalles de bajo nivel en UML al modelar las llamadas internas del motor gráfico de Canvas.	Definición de shapes como frontera del sistema en Astah, deteniendo formalmente la traza en sus métodos públicos y clarificando la delegación de responsabilidades.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

¿Qué hicimos bien como equipo? Mantuvimos una comunicación fluida, distribuimos las cargas de trabajo de forma equilibrada y adoptamos el uso riguroso de Git mediante ramas temáticas para diseño, lógica y GUI, lo que previno colisiones de código. Asimismo, fuimos disciplinados con la documentación JavaDoc y el apego a las convenciones de programación orientada a objetos.

¿A qué nos comprometemos para mejorar? Reconocemos que en este ciclo dependimos de pruebas manuales en el banco de objetos de BlueJ para verificar el sistema. Nos comprometemos a adoptar desde el

comienzo del Ciclo 2 el desarrollo guiado por pruebas unitarias automatizadas con JUnit, diseñando escenarios formales de prueba antes de dar por culminada cualquier funcionalidad.

RETROSPECTIVA CICLO 2: REFACTORING, EXTENSIÓN Y PRUEBAS UNITARIAS

Alcance: Extensión funcional (swap, lock/unlock, spin parametrizado, set), animación paso a paso y suite de pruebas unitarias automatizadas en modo invisible.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Para el Ciclo 2 ('Refactoring y Extensión'), el objetivo se centró en incorporar operaciones avanzadas de manipulación física de ruedas (intercambio, bloqueo/desbloqueo, rotación por pasos y configuración forzada), implementar la animación paso a paso en modo visible y construir una suite rigurosa de pruebas unitarias automatizadas en modo invisible. Con el propósito de no introducir regresiones sobre el código previo, definimos cuatro mini-ciclos estructurados:

Mini-ciclo	Nombre	Alcance / Requisitos	Justificación Técnica y de Diseño
MC 2.1	Refactorización y Diseño de Extensión en Astah	Actualización del diagrama de clases para incorporar swap, lock, unlock, spin(wheel, steps) y spin(setSymbols), junto con sus diagramas de secuencia dinámicos.	Planificar en Astah el impacto de las nuevas operaciones antes de modificar el código fuente, asegurando la preservación de la alta cohesión y evitando la degradación estructural de la clase SlotMachine.
MC 2.2	Implementación de Extensiones Funcionales	Codificación de swap(wheel1, wheel2), lock(wheel), unlock(wheel) y las nuevas sobrecargas de giro. Validación de invariantes de ruedas bloqueadas.	Desarrollar la lógica transaccional en memoria garantizando invariantes estrictas: una rueda bloqueada debe rechazar cualquier rotación y el intercambio de ruedas debe mantener la integridad de los índices y referencias.
MC 2.3	Visualización Dinámica y Animación Paso a Paso (Usabilidad)	Animación secuencial paso a paso en spin(wheel, steps) con temporización en Canvas cuando la máquina está visible, manteniendo la ejecución atómica en modo invisible.	Cumplir con la exigencia de usabilidad sin penalizar el desempeño del simulador en ejecuciones desatendidas. La animación debía ser un recurso didáctico visible, pero inexistente en memoria para no ralentizar pruebas.

MC 2.4	Pruebas Unitarias Automatizadas	Construcción de la suite SlotMachineC2Test en modo invisible ('qué debería hacer' vs 'qué no debería hacer'), aporte de casos en SlotMachineCC2Test y diseño de pruebas de aceptación.	Garantizar la verificación continua del sistema mediante pruebas automatizadas rápidas y determinísticas, cubriendo escenarios nominales y casos de borde (ruedas bloqueadas, parámetros negativos, índices fuera de rango).
--------	---------------------------------	--	--

En síntesis: La separación deliberada entre la animación gráfica (MC 2.3) y la infraestructura de pruebas unitarias (MC 2.4) constituyó la decisión metodológica más acertada del ciclo, permitiendo que la suite de pruebas se ejecutara en milisegundos sin verse frenada por temporizadores visuales.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

Al cierre del Ciclo 2, el proyecto reporta un 100% de cumplimiento en todos sus mini-ciclos planificados. Las nuevas funcionalidades se integraron sin fricciones y las pruebas unitarias automatizadas superaron todas las validaciones:

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 2.1	Diseño Astah	80% Completado	Diagrama de clases consolidado con las nuevas operaciones; diagramas de secuencia modelados formalmente en UML con fragmentos combinados, sin embargo al igual que en el primer ciclo, detalles de los metodos nuevos, tienen que ser pulidos
MC 2.2	Lógica	100% Completado	Métodos swap, lock, unlock y variantes sobrecargadas de spin plenamente funcionales y con validación de precondiciones.
MC 2.3	Animación Paso a Paso	100% Completado	spin(wheel, steps) muestra la transición incremental frame a frame cuando isVisible() es verdadero, y ejecuta la rotación instantánea cuando es falso.

MC 2.4	Pruebas Unitarias & Aceptación	80% Completado	Suite SlotMachineC2Test en verde al 100% en modo invisible; pruebas correspondientes que aseguran el buen funcionamiento de los metodos
--------	--------------------------------	----------------	---

¿Por qué se encuentra en este estado? Porque se cumplieron a cabalidad los requisitos de entrega de la Semana 06. El simulador no solo amplió su espectro de operaciones mecánicas, sino que incorporó una red de seguridad basada en pruebas unitarias que blindó al sistema contra regresiones.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

En el Ciclo 2 mantuvimos una inversión de 8 Horas/Hombre por integrante (16 Horas/Hombre acumuladas en el ciclo). El tiempo se distribuyó atacando en paralelo la refactorización lógica y el diseño del arnés de pruebas:

Desarrollador	Horas	Responsabilidades y Foco de Trabajo	Rol en el Ciclo
Juan David Espitia	8 Horas	Refactorización de la clase Wheel para soportar estados de bloqueo (lock/unlock), desarrollo de swap, modelado de secuencias en Astah y diseño de casos de error en SlotMachineC2Test.	Lógica y Testing
Cristian Salamanca	8 Horas	Implementación del motor de animación paso a paso en spin(wheel, steps), temporización sobre Canvas, integración de casos en SlotMachineCC2Test y pruebas de aceptación.	Animación y Usabilidad

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Consideramos que el mayor logro del Ciclo 2 fue haber implementado una dualidad operativa transparente en el método spin(wheel, steps), logrando que ofrezca una experiencia visual enriquecida con animación suave paso a paso cuando la máquina está en modo visible, mientras que en modo invisible ejecuta la transición de estados de forma instantánea y atómica en memoria.

¿Por qué lo consideramos el mayor logro? Porque resolver la sincronización temporal entre la interfaz gráfica y la lógica del modelo sin degradar el rendimiento es uno de los problemas formativos más desafiantes. Si hubiésemos dejado la animación activa de forma indiscriminada, la ejecución de la suite de pruebas unitarias habría tardado minutos; si hubiésemos omitido la animación, no habríamos cumplido la usabilidad. Lograr que el mismo método distinga su contexto de visibilidad permitió que SlotMachineC2Test ejecutara decenas de casos en milisegundos, demostrando la solidez y madurez de nuestro diseño orientado por objetos.

■ SÍNTESIS DEL MAYOR LOGRO - CICLO 2

Dualidad transparente en spin(wheel, steps): animación paso a paso en modo visible y ejecución atómica instantánea en modo invisible.

Suite SlotMachineC2Test 100% automatizada evaluando caminos de éxito ('qué debería hacer').

Aporte colaborativo a SlotMachineCC2Test y preparación de casos de aceptación para sustentación.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

Los mayores problemas técnicos afrontados en el Ciclo 2 se manifestaron en la reutilización de código frente a las sobrecargas de giro y en la complejidad del modelado de Diagramas de Secuencia en Astah bajo estructuras iterativas y condicionales.

Respecto a la reutilización de código, al introducir cuatro variantes de giro (spin(), spin(wheel), spin(wheel, steps), spin(setSymbols)), inicialmente identificamos una fuerte duplicación de lógica: cada método intentaba recalcular los desplazamientos circulares y verificar de manera dispersa si la rueda objetivo estaba bloqueada (lock). Esta redundancia amenazaba la mantenibilidad. Para resolverlo, aplicamos una refactorización modular: extrajimos la rotación elemental de una posición hacia un método protegido en Wheel (rotateOneStep()), el cual valida de forma atómica el estado de bloqueo antes de alterar los punteros de símbolo. De este modo, todas las sobrecargas de spin reutilizan este método central en bucle, eliminando la redundancia y centralizando el manejo de precondiciones.

Respecto a los Diagramas de Secuencia en Astah, modelar spin(wheel, steps) representó un salto cualitativo de complejidad respecto al ciclo anterior. El flujo ya no era secuencial, sino que requería fragmentos combinados de UML: un bloque condicional (alt) para verificar si la rueda estaba bloqueada, un bloque iterativo (loop) para representar las k iteraciones del avance, y un bloque opcional interno (opt) para invocar la animación en shapes únicamente si isVisible() era verdadero. Para resolverlo, estudiamos en detalle la notación formal de fragmentos combinados en Astah, estructurando un diagrama riguroso que refleja con precisión las condiciones booleanas de guarda sin sacrificar la legibilidad arquitectónica.

Dimensión Técnica	Desafío y Dificultad Afrontada	Estrategia de Solución Aplicada
Reutilización de Código (Sobrecarga de Giros y Lock)	Riesgo de lógica redundante en el avance circular de símbolos y validación dispersa de ruedas bloqueadas a través de 4 variantes de spin.	Refactorización hacia un método atómico rotateOneStep() en Wheel que centraliza la validación de bloqueo y el desplazamiento modular, reutilizado por todas las variantes de spin.

Diagramas de Secuencia (Asth con Loops y Condicionales)	Modelar en UML la ejecución iterativa paso a paso y la bifurcación visible/invisible sin saturar el diagrama ni vulnerar la notación formal.	Aplicación de fragmentos combinados en Astah: un alt para la verificación de bloqueo, un loop para la iteración de pasos y un opt subordinado a la bandera isVisible.
---	--	---

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

¿Qué hicimos bien como equipo? La adopción rigurosa de las prácticas de pruebas unitarias (XP/TDD). Diseñar pruebas considerando activamente las preguntas '¿qué debería hacer?' y '¿qué no debería hacer?' nos permitió anticipar escenarios anómalos (parámetros negativos, ruedas bloqueadas, índices fuera de rango) antes de proceder con la integración visual. Asimismo, la colaboración en la wiki compartida siguiendo la convención accordingEsSaShould... se desarrolló sin contratiempos.

¿A qué nos comprometemos para mejorar? De cara al Ciclo 3 nos enfrentaremos al núcleo del problema competitivo: resolver la Maratón Mundial ICPC (Problem I: Slot Machine). Nos comprometemos a realizar una exhaustiva investigación matemática y algorítmica previa, modelando la interacción en papel antes de codificar la clase SlotMachineContest, asegurando un estricto desacoplamiento frente a la testing tool.

RETROSPECTIVA CICLO 3: SOLUCIÓN MARATÓN MUNDIAL ICPC Y SIMULACIÓN VISUAL

Alcance: Desacoplamiento de SlotMachine como testing tool de caja negra y simulador, resolución interactiva del Problem I (ICPC 2025) y simulación paso a paso.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

En el Ciclo 3 abordamos el desafío central del proyecto: solucionar el Problem I de la 49ª Maratón Mundial de Programación ICPC 2025 mediante la clase SlotMachineContest. El enunciado impuso una dualidad estricta para SlotMachine: fungir como testing tool de caja negra en solve(n) (interactuando exclusivamente con spin y distinctSymbols en modo invisible sin exceder 10.000 acciones), y actuar como simulador visual en simulate(n). Con el fin de coordinar este reto algorítmico y de diseño, estructuramos el desarrollo en cinco mini-ciclos:

Mini-ciclo	Nombre	Alcance / Requisitos	Justificación Técnica y de Diseño
MC 3.1	Arquitectura y Diseño en Astah (Solver vs. Testing Tool)	Modelado de la clase controladora SlotMachineContest y su relación de uso/asociación con SlotMachine en Astah. Delimitación de contratos.	Establecer formalmente las fronteras de comunicación. Era imperativo garantizar desde el diseño que el solucionador interactuara como un cliente externo ciego, sin acceso a atributos privados o configuraciones internas de la máquina.

MC 3.2	Extensión del Simulador para Maratón	Constructor SlotMachine(n) con n ruedas y n símbolos aleatorios, y método distinctSymbols().	Suministrar la infraestructura requerida por el problema competitivo: generar una configuración aleatoria no ganadora y proveer la consulta eficiente de la cantidad k de símbolos distintos actualmente visibles.
MC 3.3	Algoritmo de Solución ICPC	Implementación de solve(n): int[][] en SlotMachineContest. Estrategia interactiva para alcanzar Jackpot (k=1) en <= 10.000 acciones.	El corazón algorítmico del ciclo: deducir la alineación de las ruedas a ciegas, basándose únicamente en cómo se incrementa o decrementa k tras cada rotación, sin conocer los colores o símbolos en pantalla.
MC 3.4	Simulación Visual de la Secuencia Ganadora	Implementación de simulate(n) en SlotMachineContest, ejecutando paso a paso sobre la máquina visible la secuencia calculada por solve.	Articular la solución matemática con la interfaz gráfica construida en los ciclos 1 y 2, permitiendo al usuario apreciar cómo la secuencia calculada transiciona ordenadamente hasta el estado ganador (Jackpot).
MC 3.5	Verificación Sistemática, Casos de Estrés y Pruebas Compartidas	Suite SlotMachineContestTest, pruebas de estrés con n limite adaptado al proyecto, colaboración en SlotMachineContestCCTest 2 y pruebas de aceptación.	Verificar formalmente la convergencia infalible del algoritmo ante cualquier configuración aleatoria inicial y comprobar que el total de jugadas se mantiene holgadamente por debajo de la cota de 10.000 acciones.

En síntesis: Esta partición modular en cinco mini-ciclos protegió la lógica de competencia de la capa de presentación. Resolvimos primero la deducción matemática interactiva (MC 3.3) y, una vez asegurada la convergencia, integramos la simulación visual (MC 3.4), evitando contaminar la arquitectura con dependencias cruzadas.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

El estado actual del proyecto en el Ciclo 3 reporta un 98% de avance consolidado. Los componentes principales de resolución algorítmica y simulación visual se encuentran plenamente operativos, restando únicamente la afinación de casos extremos de sustentación:

Mini-ciclo	Nombre	Estado	Evidencias y Verificación del Estado
MC 3.1	Diseño Astah	90% Completado	Diagrama de clases y diagramas de secuencia para slotMachine de forma robusta y detallada con detalles minimos pendientes
MC 3.2	Extensión RF13	100% Completado	Constructor aleatorio SlotMachine(n) y método distinctSymbols() implementados y verificados en modo invisible.
MC 3.3	Algoritmo solve(n)	100% Completado	Estrategia interactiva de alineación operativa; retorna la matriz int[][] que garantiza Jackpot dentro del límite de 10.000 acciones.
MC 3.4	Simulador simulate(n)	100% Completado	simulate(n) orquesta la resolución ciega y reproduce la animación paso a paso sobre Canvas hasta cantar Jackpot visual.
MC 3.5	Pruebas Contest	90% En Progreso	SlotMachineContestTest aprobada en verde.

¿Por qué se encuentra en este estado? Porque el núcleo algorítmico y de ingeniería del ciclo está 100% resuelto. El 2% restante corresponde a la consolidación de las pruebas colaborativas compartidas y al ensayo final de las pruebas de aceptación preparadas para la sustentación ante el docente.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)

En este ciclo mantuvimos una inversión balanceada de 8 Horas/Hombre por integrante (16 Horas/Hombre totales para el Ciclo 3). La dedicación se enfocó estratégicamente en los dos pilares del problema:

Desarrollador	Horas	Responsabilidades y Foco de Trabajo	Rol en el Ciclo
Juan David Espitia	8 Horas	Investigación y formulación del algoritmo interactivo en <code>SlotMachineContest.solve(n)</code> , optimización de la cota de acciones generadas y construcción de la suite de pruebas en <code>SlotMachineContestTest</code> .	Algoritmia y Resolución ICPC
Cristian Salamanca	8 Horas	Implementación de la infraestructura en <code>SlotMachine</code> (<code>SlotMachine(n)</code> , <code>distinctSymbols()</code>), desarrollo de la simulación visual en <code>simulate(n)</code> , diagramación en Astah y pruebas de aceptación.	Testing Tool, GUI y Astah

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Consideramos que el mayor logro del Ciclo 3 fue haber solucionado con éxito un problema de nivel de Maratón Mundial ICPC manteniendo un diseño orientado a objetos impecable y respetando con absoluta rigurosidad el principio de caja negra.

¿Por qué lo consideramos el mayor logro? Porque en la programación competitiva clásica es habitual resolver problemas interactivos mediante scripts planos con variables globales. Integrar este desafío dentro de un diseño de software estructurado —donde la clase `SlotMachine` actúa como un evaluador silencioso que solo expone `spin` y `distinctSymbols`, y donde esa misma solución matemática se convierte de inmediato en una simulación visual sincronizada paso a paso mediante `simulate(n)`— demostró la solidez de los principios de ingeniería de software aprendidos en la cátedra. Demostramos que la disciplina de diseño no obstaculiza la algoritmia compleja, sino que la vuelve comprensible, verificable y extensible.

■ SÍNTESIS DEL MAYOR LOGRO - CICLO 3

Resolución interactiva del Problem I (ICPC World Finals 2025) a ciegas dentro del tope de 10.000 acciones.

Desacoplamiento total: `SlotMachine` actúa como testing tool silenciosa en `solve(n)` y como simulador estético en `simulate(n)`.

Reutilización integral de la base de código de los ciclos 1 y 2 sin alterar su estructura.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico del Ciclo 3 radicó en la reutilización de código bajo restricciones estrictas de caja negra y en la elaboración de Diagramas de Secuencia para un protocolo algorítmico interactivo y no determinista.

En cuanto a la reutilización de código, el reto fundamental consistió en que la clase `SlotMachine` debía cumplir dos roles conceptualmente opuestos: por un lado, debía actuar como testing tool para que el solver interactuara de forma ciega (sin acceso a símbolos ni colores), y por otro, debía seguir siendo el simulador gráfico interactivo para `simulate(n)`. Existía el riesgo de violar el encapsulamiento o permitir que el solver inspeccionara atributos privados para simplificar la solución. Para resolverlo, aplicamos encapsulamiento riguroso mediante composición: `SlotMachineContest` no accede a ningún campo interno de `SlotMachine`; en `solve(n)` consume exclusivamente la interfaz autorizada en modo invisible, y en `simulate(n)` reutiliza la matriz de acciones calculada para invocar `makeVisible()` y ejecutar los giros paso a paso sobre el motor gráfico desarrollado en el Ciclo 2.

En cuanto a los Diagramas de Secuencia en Astah, modelar `solve(n)` representó el desafío UML más exigente del semestre. A diferencia de un método determinista lineal, `solve(n)` ejecuta un proceso interactivo de toma de decisiones: gira una rueda, consulta `distinctSymbols()`, analiza la variación de `k` y deduce dinámicamente la siguiente acción dentro de un bucle hasta alcanzar `k=1`. Modelar esta interacción dinámica en Astah sin incurrir en diagramas sobrecargados exigió diseñar dos diagramas separados: uno para `solve(n)` con un fragmento combinado loop condicionado a `[distinctSymbols > 1]` alternando llamadas a `spin` y consultas de `k`; y otro independiente para `simulate(n)` que muestra la iteración secuencial sobre la matriz de acciones delegando el renderizado hacia shapes.

Dimensión Técnica	Desafío y Dificultad Afrontada	Estrategia de Solución Aplicada
Reutilización de Código (Testing Tool vs. Simulador)	Reutilizar <code>SlotMachine</code> para evaluar al solver sin filtrar datos prohibidos de los símbolos y habilitar su animación visual en <code>simulate</code> .	Aislamiento estricto por composición: <code>solve(n)</code> interactúa con la interfaz restringida en modo invisible y <code>simulate(n)</code> reutiliza la matriz devuelta por <code>solve</code> para animar los giros en pantalla con <code>makeVisible()</code> .
Diagramas de Secuencia (Astah para Algoritmo Interactivo)	Representar en UML un algoritmo interactivo que decide sus acciones dinámicamente según la retroalimentación del oráculo <code>distinctSymbols()</code> .	Elaboración de dos diagramas en Astah: uno para <code>solve(n)</code> con fragmento loop <code>[k > 1]</code> y consultas alternadas a <code>distinctSymbols</code> , y otro para <code>simulate(n)</code> mostrando la delegación visual secuencial sobre shapes.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

¿Qué hicimos bien como equipo? Destacamos la rigurosidad analítica y la disciplina metodológica. En lugar de precipitarnos a programar heurísticas empíricas, dedicamos sesiones iniciales a simular la máquina en papel y lápiz con casos reducidos (`n=3` y `n=5`), comprendiendo con exactitud qué información revela cada cambio en

el conteo de símbolos distintos k . Esto nos permitió diseñar un algoritmo determinístico y evitó callejones sin salida. Además, la sinergia entre la lógica algorítmica de Juan David y el soporte visual de Cristian funcionó de forma óptima.

¿A qué nos comprometemos para mejorar? De cara al Ciclo 4 (entrega final y sustentación), nos comprometemos a optimizar la heurística para reducir la cota promedio de jugadas, ejecutar pruebas de estrés exhaustivas en el límite máximo de la competencia ($n=50$) y pulir la presentación ejecutiva para cerrar la asignatura con la máxima calificación posible.